Que la transferencia no deje dinero en el aire

Saga orquestada y circuit breaker en Mexi Banco

Pablo Vaquero
Cuenta: 318259624
Sistemas Distribuidos 2027-1 · Grupo 2
15 de octubre de 2026 · Ciudad de México

La transferencia guarda su avance y devuelve el cargo cuando el abono es rechazado de forma confirmada. Si no puede devolverlo, conserva un estado pendiente. Un circuit breaker manual protege todas las llamadas de transferencia a cuenta.

Resultados de la ejecución

Prueba	Resultado observado
Versión original	Origen 800.00, destino 500.00; faltan 200.00 en la suma.
Transferencia normal	HTTP 201, COMPLETADA; origen 800.00 y destino 700.00.
Abono rechazado	HTTP 409, COMPENSADA; origen 1,000.00 y destino 500.00.
Cuenta apagada	HTTP 202, COMPENSACION_PENDIENTE; persiste tras reiniciar.
Circuito	CERRADO → ABIERTO → SEMIABIERTO → CERRADO.
Pruebas Java	25 pruebas aprobadas, sin fallas ni errores.

Base y alcance

Base oficial: rama 03-gateway-discovery-balanceo, commit 87cfbe97. El entorno usa Java 21, PostgreSQL 17, Eureka y gateway. Se ejecutó con una réplica de cuenta, permitida por el enunciado. Los cambios funcionales están en transferencia y en docker-compose.yml.

Las capturas son vistas de las respuestas HTTP, consultas SQL y logs obtenidos durante estas ejecuciones. Los archivos originales y el script de reproducción acompañan la entrega en evidencia/. Las marcas terminadas en Z están en UTC; la ejecución local fue el 15 de octubre.

1. El problema antes del cambio

Se crearon dos cuentas por el gateway con 1,000.00 y 500.00. Se configuró una pausa de 20 segundos entre cargo y abono. Después de observar el cargo de 200.00, se detuvo cuenta; al recuperarla, el destino seguía sin recibir el monto.

```
PAUSA_ENTRE_CARGO_Y_ABONO_MS=20000 docker compose up -d transferencia
# POST http://localhost:8080/api/transferencias, monto: 200
docker compose stop cuenta
docker compose start cuenta
```



Parte 0 · El cargo quedó aplicado

2026-10-15T18:11:07.466028-06:00

Saldo	Antes	Después
Origen	1,000.00	800.00
Destino	500.00	500.00
Suma	1,500.00	1,300.00

POST /api/transferencias · monto 200.00 · pausa 20 s

```
HTTP 504 Gateway Timeout
Consulta SQL: tabla transferencia → 0 filas
```

Captura de resultados ejecutados · Los archivos JSON y logs originales se conservan en evidencia/.

Figura 1. Antes y después de la Parte 0. Fuente: 01-problema-original.json.

Por qué ocurrió

El cargo se confirmó en la base de cuenta antes de intentar el abono. Una transacción local de transferencia no puede deshacer una transacción ya confirmada en otro servicio. Además, la versión original guardaba la fila de transferencia al final; al fallar antes, la tabla quedó vacía.

El gateway original respondió 504 a los 15 segundos, antes de terminar la pausa. Ese timeout no canceló el cargo. Para observar la respuesta final de la saga en la prueba con pausa, se aumentó su límite a 45 segundos mediante una variable de Compose. Las llamadas salientes conservan límites de 2 segundos de conexión y lectura.

2. La saga y sus estados

TransferenciaService decide qué paso sigue. Primero persiste INICIADA. Consulta las cuentas, carga el origen, abona el destino, registra los movimientos y envía los avisos. Cada guardado tiene una transacción local independiente.

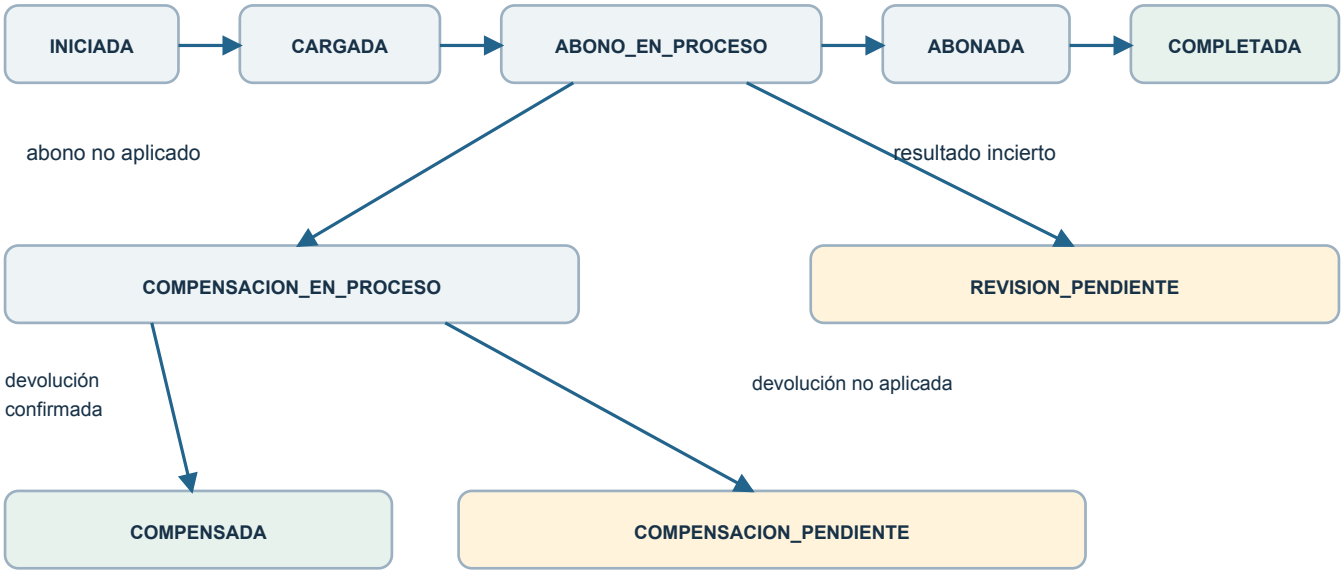


Figura 2. Rutas principales, con pasos intermedios resumidos. Un resultado incierto del cargo o de la devolución también lleva a REVISION_PENDIENTE. Los estados adicionales se describen abajo.

Paso	Efecto y respuesta a una falla
Consultar cuentas	No modifica dinero. Un rechazo termina RECHAZADA; una caída, NO_DISPONIBLE.
Cargar origen	Se guarda CARGO_EN_PROCESO antes de enviar y CARGADA al confirmar. Su compensación es abonar el mismo monto al origen.
Abonar destino	Se guarda ABONO_EN_PROCESO y después ABONADA. Si se confirma el rechazo, se devuelve el cargo. Si el resultado es incierto, se requiere revisión.
Devolver al origen	COMPENSACION_EN_PROCESO → COMPENSACION_APLICADA → COMPENSADA. Si no se puede enviar: COMPENSACION_PENDIENTE.
Movimientos y avisos	Con el dinero ya confirmado, un fallo del ledger deja REGISTRO_PENDIENTE. Un aviso fallido conserva COMPLETADA y lo explica en detalle. No devuelve dinero de nuevo.

El historial y los estados previos a cada llamada permiten localizar una caída entre el envío y la confirmación. Persistir el estado permite investigar y recuperar; esta entrega no incorpora un proceso automático de recuperación.

3. El código clave

Guardar antes y después de cada efecto

```
t = guardar(t, CARGO_EN_PROCESO, "Se enviara el cargo al origen");
CuentaRemota cargada = cuentas.cargar(origen, monto);
t = guardar(t, CARGADA, "Cargo confirmado; el abono todavia no ocurre");
```

Se muestra el flujo resumido; el archivo incluye el manejo de errores alrededor de la llamada. guardar() actualiza estado, detalle, fecha e historial, y llama a saveAndFlush(). TransferenciaService no lleva una transacción que envuelva toda la saga: cada llamada al repositorio confirma antes de continuar.

Compensar solo con un rechazo confirmado

```
if (!noAplicada(e)) {
    return guardar(t, REVISION_PENDIENTE,
        "Cargo confirmado y abono incierto. Conciliar antes de devolver");
}
return compensar(t, cargada, e);
```

Un rechazo 404, 409 o 422 del contrato de cuenta, un circuito abierto o una conexión rechazada permiten saber que no se aplicó el abono. Un timeout de lectura o un error 500 puede ocurrir después del commit: devolver automáticamente en ese caso podría crear dinero. Por eso se conserva REVISION_PENDIENTE.

La devolución usa la misma protección

```
devuelta = cuentas.abonar(t.getClaveOrigen(), t.getMonto());
// Después se persiste COMPENSACION_APLICADA y se registran los asientos.
movimientos.registrar(t.getClaveOrigen(), TipoMovimiento.TRANSFERENCIA_RECIBIDA,
    t.getMonto(), devuelta.saldo(), "COMPENSACION saga " + t.getId() + ...);
```

La compensación usa TRANSFERENCIA_RECIBIDA y una referencia que identifica la saga; así se mantiene el contrato del servicio movimiento sin agregarle otro enum. La fila del cargo usa TRANSFERENCIA_ENVIADA. Si falla el registro después de devolver, queda REGISTRO_PENDIENTE y no se repite la devolución.

Circuit breaker y fallas controladas

```
return circuito.llamar(() -> http.post().uri("/cuentas/{clabe}/cargos", clabe)
    .body(Map.of("monto", monto)).retrieve().body(CuentaRemota.class));
```

Consultar, cargar y abonar pasan por CircuitoCuenta. FALLAR_ABONO_CLABE provoca un 422 local antes de enviar el abono a esa CLABE; se aplica solo al abono, por lo que el cargo y la devolución a otra cuenta siguen funcionando. Esta simulación no cuenta como falla de infraestructura.

Archivos principales: TransferenciaService.java, Transferencia.java, EstadoTransferencia.java, CuentaCliente.java, CircuitoCuenta.java y TransferenciaController.java. Los fragmentos anteriores están abreviados para explicar el flujo.

4. La compensación funcionando

Se activó FALLAR_ABONO_CLABE=002180000000008202. Con cuenta encendida, la transferencia 1 cargó 200.00 al origen 002180000000008201 y el abono al destino fue rechazado antes de enviarse. La saga devolvió el monto al origen.

Saga · Dinero devuelto y asentado

2026-10-15T18:13:54.264640-06:00

Saldo	Antes	Después
Origen	1,000.00	1,000.00
Destino	500.00	500.00
Suma	1,500.00	1,500.00

> HTTP 409 · ID 1 · COMPENSADA

El abono fallo y el dinero fue devuelto al origen; movimientos registrados

Consulta a la base movimiento

CLABE (final)	Tipo	Monto	Saldo	Referencia
008201	DEPOSITO	1000.00	1000.00	saldo inicial
008202	DEPOSITO	500.00	500.00	saldo inicial
008201	TRANSFERENCIA_ENVIADA	-200.00	800.00	saga 1 a 002180000000008202
008201	TRANSFERENCIA_RECIBIDA	200.00	1000.00	COMPENSACION saga 1: abono destino no aplicado

Captura de resultados ejecutados · Los archivos JSON y logs originales se conservan en evidencia/.

Figura 3. Saldos, respuesta y movimientos consultados en PostgreSQL. Fuente: 03-compensacion.json.

La consulta a la tabla transferencia confirmó COMPENSADA. El historial guardado conserva INICIADA, CARGO_EN_PROCESO, CARGADA, ABONO_EN_PROCESO, COMPENSACION_EN_PROCESO, COMPENSACION_APLICADA y COMPENSADA. El cliente recibió HTTP 409 con el ID, el estado y la explicación de la devolución.

También se probó una CLABE no incluida en la falla: la transferencia 2 terminó COMPLETADA con HTTP 201, origen 800.00 y destino 700.00. Su evidencia completa está en 02-transferencia-exitosa.json.

5. Si la devolución también falla

Con otra pareja de cuentas se repitió la pausa de 20 segundos. Durante la pausa, PostgreSQL mostraba CARGADA y el origen tenía 800.00. Al apagar cuenta, no había instancia disponible ni para abonar al destino ni para devolver al origen.



Saga · Compensación pendiente y durable

2026-10-15T18:15:12.190197-06:00

Saldo	Antes	Después
Origen	1,000.00	800.00
Destino	500.00	500.00
Suma	1,500.00	1,300.00

HTTP 202 · ID 3 · COMPENSACION_PENDIENTE

Durante la pausa: saldo de origen 800.00.

```
 id | estado  |                  detalle
----+---------+-----------------------------------------------
  3 | CARGADA | Cargo confirmado; el abono todavia no ocurre
(1 row)
```

Después de reiniciar transferencia: **COMPENSACION_PENDIENTE**.

Abono destino no aplicado; devolucion sin confirmar. Conciliar antes de reintentar. No instances available for cuenta

Captura de resultados ejecutados · Los archivos JSON y logs originales se conservan en evidencia/.

Figura 4. La transferencia 3 conserva COMPENSACION_PENDIENTE después de reiniciar el servicio.

Cómo terminaría la compensación

Primero se concilian los efectos remotos con un identificador de operación. Si se confirma que el destino no recibió el dinero y que no hubo devolución, se reintenta únicamente la devolución pendiente. No se vuelve a ejecutar la transferencia completa. Tras confirmar la devolución, se registran los movimientos pendientes y se avanza a COMPENSADA.

Para automatizarlo agregaría idempotencia en cuenta: la llave idSaga:compensacion, el cambio de saldo y la respuesta se guardan juntos en una transacción local con restricción única. Un trabajador podría reintentar con esperas de 1, 2, 4 segundos y un límite. Si la respuesta del primer intento se pierde, la misma llave recuperaría el resultado sin abonar dos veces.

Límite de esta entrega: la API original de cuenta no admite esa llave. No se añadió reintento automático ni se promete idempotencia de extremo a extremo. HTTP 202 comunica que el caso sigue pendiente de conciliación; el servicio no lo termina por sí solo.

6. Los tres estados del circuito

El circuit breaker está implementado a mano. CERRADO permite llamadas y cuenta fallas consecutivas de infraestructura. Con tres fallas pasa a ABIERTO. Después de 15 segundos, la siguiente llamada es la única prueba SEMIABIERTA; si responde, vuelve a CERRADO; si falla, reinicia la espera.

Circuit breaker · Apertura y recuperación

2026-10-15T18:15:44.308226-06:00

Momento	Estado	Llamadas permitidas	Rechazadas
Inicial	CERRADO	0	0
Tras fallas	ABIERTO	3	0
Tres rechazos rápidos	ABIERTO	3	3
Recuperado	CERRADO	7	3

Solicitud con circuito abierto	HTTP	Duración
1	503	14.71 ms
2	503	14.01 ms
3	503	10.39 ms

```
2026-10-16T00:15:20.821Z CIRCUITO cuenta CERRADO -> ABIERTO
2026-10-16T00:15:43.897Z CIRCUITO cuenta ABIERTO -> SEMIABIERTO
2026-10-16T00:15:43.932Z CIRCUITO cuenta SEMIABIERTO -> CERRADO
```

Captura de resultados ejecutados · Los archivos JSON y logs originales se conservan en evidencia/.

Figura 5. Contadores, duraciones por el gateway y transiciones extraídas de los logs.

Con cuenta apagada se permitieron tres intentos. Las siguientes tres solicitudes respondieron 503 en 14.71, 14.01 y 10.39 ms; el contador de llamadas permitidas se mantuvo en 3. Al recuperar cuenta, los logs registraron SEMIABIERTO y CERRADO, y una nueva transferencia de 1.00 terminó correctamente.

La admisión es sincronizada, pero no mantiene el bloqueo durante HTTP. Un número de generación impide que una respuesta antigua cierre un circuito que otra llamada acaba de abrir. Los 404 y 422 no se consideran caídas; una respuesta de negocio demuestra que el servicio respondió. El estado del circuito es local a cada instancia y se reinicia al arrancarla.

```
GET http://localhost:8080/api/transferencias/circuito
CUENTA_CB_UMBRAL=3       CUENTA_CB_ESPERA_MS=15000
HTTP_CONEXION_MS=2000    HTTP_LECTURA_MS=2000
```

7. Aislamiento y validación

Lo que la saga no garantiza

Entre el cargo y la compensación, otra consulta sí ve el saldo reducido de 800.00, aunque al final vuelva a 1,000.00. Ese valor intermedio ya está confirmado en cuenta. Una segunda operación podría rechazar un cargo por saldo insuficiente o tomar una decisión usando ese saldo. La compensación no borra lo que otros observaron ni deshace automáticamente sus decisiones. Una saga permite recuperar consistencia mediante operaciones de negocio, pero no proporciona el aislamiento de una transacción global.

Pruebas ejecutadas



Figura 6. Resumen de Maven; ejecución dentro de Java 21 en Docker.

Las pruebas cubren éxito, rechazo del abono, devolución única, compensación pendiente, timeouts ambiguos, fallo del ledger y de avisos, montos inválidos, tres estados del circuito, una sola prueba concurrente en semiabierto y respuestas antiguas. También conservan la verificación del balanceo por nombre entre dos réplicas HTTP simuladas.

Límites y siguientes pasos

No se implementó recuperación automática tras una caída del orquestador, deduplicación del POST de transferencia, Outbox ni registro atómico entre saldo y ledger. Se conserva evidencia para conciliar esos casos. Un estado EN_PROCESO después de una caída no autoriza repetir el efecto: puede haberse confirmado remotamente. La solución satisface la compensación demostrable del laboratorio y hace visibles los casos que necesitan revisión.

Material consultado

1. Enunciado Lab S08, main del curso, commit 2f198c45: github.com/OscarRuiz21/sd-2027-1.
2. S08-Clase-8-SD.pdf: saga y orquestación (láminas 21-30), resiliencia (35-38).
3. Fowler, CircuitBreaker: martinfowler.com/bliki/CircuitBreaker.html.
4. Richardson, Pattern: Saga: microservices.io/patterns/data/saga.html.